ADS205 Decision, Computation and Language

Lecture 6 – Pumping Lemma for Context-Free Languages & Review

Dr Chunchuan Lyu and Dr Xinhong Chen



Xi'an Jiaotong-Liverpool University
西交利物浦大学

Recap

- Pushdown Automata (PDA)
- CFG $\longleftrightarrow$ PDA

Today

- Pumping Lemma for CFLs
- Review



Recap: PDA & Equivalence to CFG

- Pushdown Automata (PDA) is used for context-free languages to judge whether a string is accepted or not.
- PDA is a stack to store the **unprocessed symbols** during derivation of strings.
- **CFG and PDA are equivalent**.



How to determine whether a language is context-free?

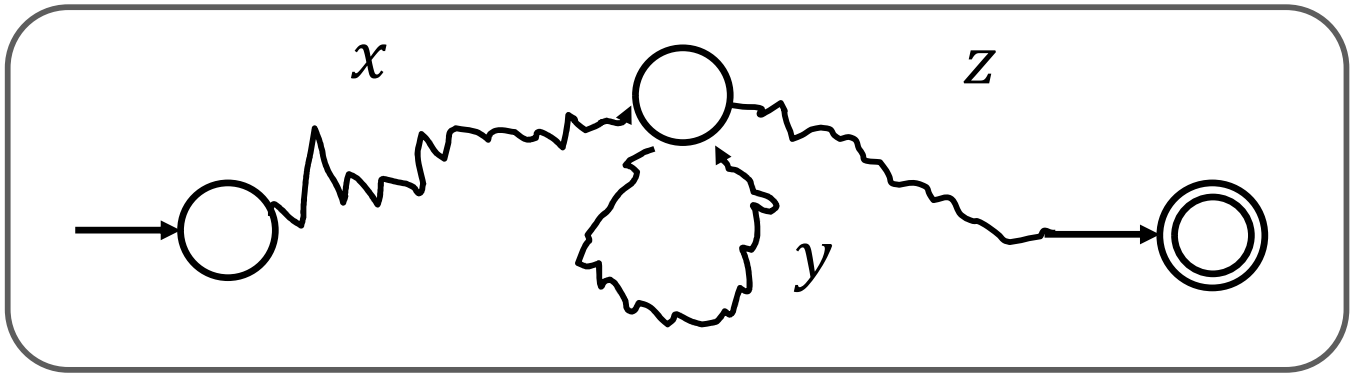
- For regular languages, we learnt FAs, regexes, and then we use pumping lemma to determine whether a language is regular.
- Can we do the same for CFLs?
- Yes => **Pumping lemma for CFLs**



Pumping lemma for CFLs

- Recall Pumping Lemma for RE

a regular language L



- a **long-enough** string $w = xyz, w \in L, y \neq \varepsilon$
- strings we could create by looping y: $\{w | w = xy^i z \ (i \geq 0)\}$
- we should have $xy^i z \in L$ for all $i \geq 0$



(Corrected) The Regular Pumping Lemma

- Example:
- Prove $L = \{0^n 1^n : n \geq 0\}$ is not regular

① Assume L is regular, then there exists a p for L …, and all $w \in L$ with $|w| \geq p$, … (The pumping lemma)

② say we choose $w = 0^p 1^p = 000 \cdots 000111 \cdots 111$

③ Check **ALL possible breakups** of $w = xyz$, such as:

$$w = \underbrace{000 \cdots 000111 \cdots 111}_{x = 0^a \;\; y = 0^b, \;\; z = 0^{p-a-b}1^p}$$

$b \geq 1$

$\boldsymbol{since\ |xy| \leq p, so\ a + b \leq p}$



The Regular Pumping Lemma

$$w = \overbrace{000\cdots000111\cdots111}$$
$$x = 0^a \quad y = 0^b, \quad z = 0^{p-a-b}1^p$$
$$b \geq 1, a + b \leq p$$

- Example:
- Prove $L = \{0^n 1^n : n \geq 0\}$ is not regular

④ Loop the repeated part to create strings $xy^i z, i \geq 0$

For example, $i = 2$, we have $xy^2z = 0^a 0^b 0^b 0^{p-a-b}1^p = 0^{p+b}1^p$

⑤ Check whether any string contradicts with L

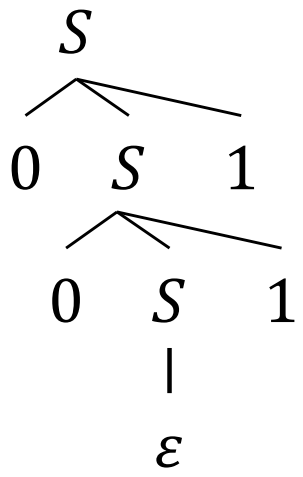
If xy^2z is in L, then $p + b = p$, which is impossible

➢ L is not regular.

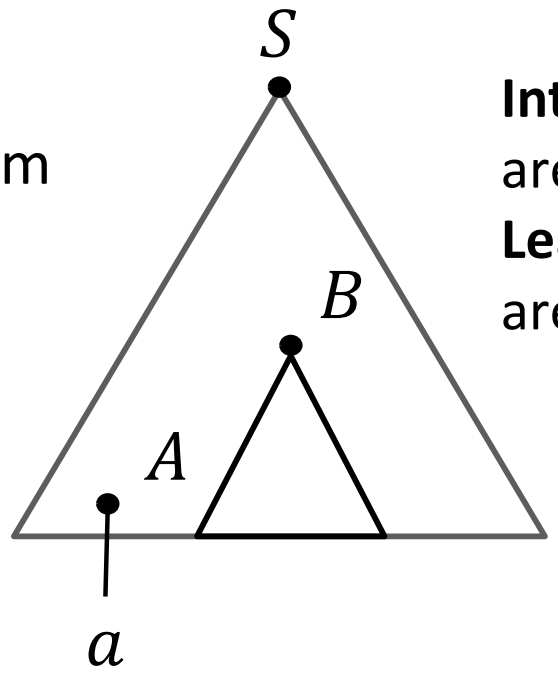


Pumping lemma for CFLs

- See if we could do something similar to CFLs
- For CFLs, we can have a parse tree for their strings, following derivation



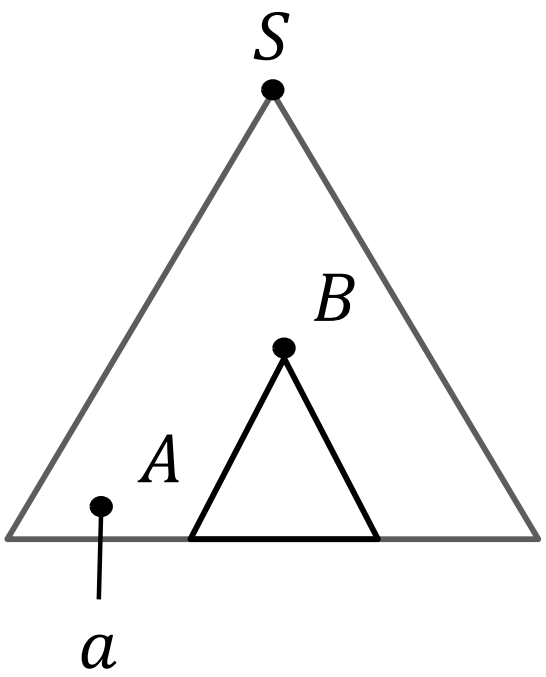
generalized form



Internal nodes are **variables**. **Leave** nodes are **terminals**.

Pumping lemma for CFLs

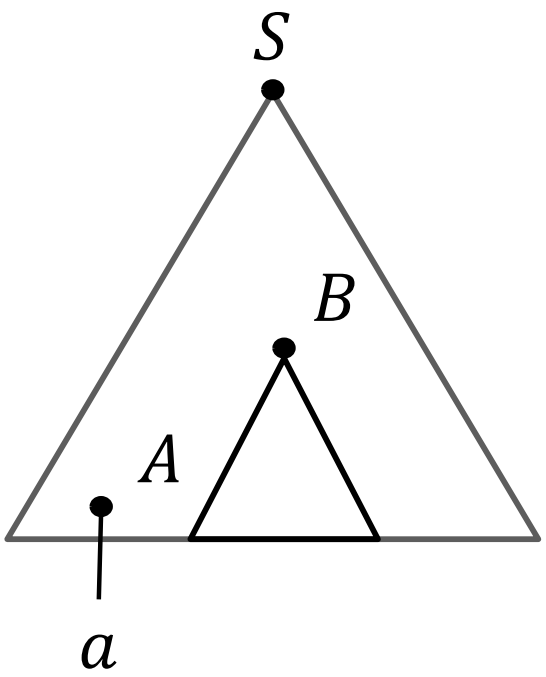
- For CFLs, we can have a parse tree for their strings, following derivation
- Internal nodes are variables, and leave nodes are terminals.



- CFGs have limited rules => we have **limited variables**
- Imagine an extreme case where **each variable is only used once** when generating strings => we only have a string that has limited length

Pumping lemma for CFLs

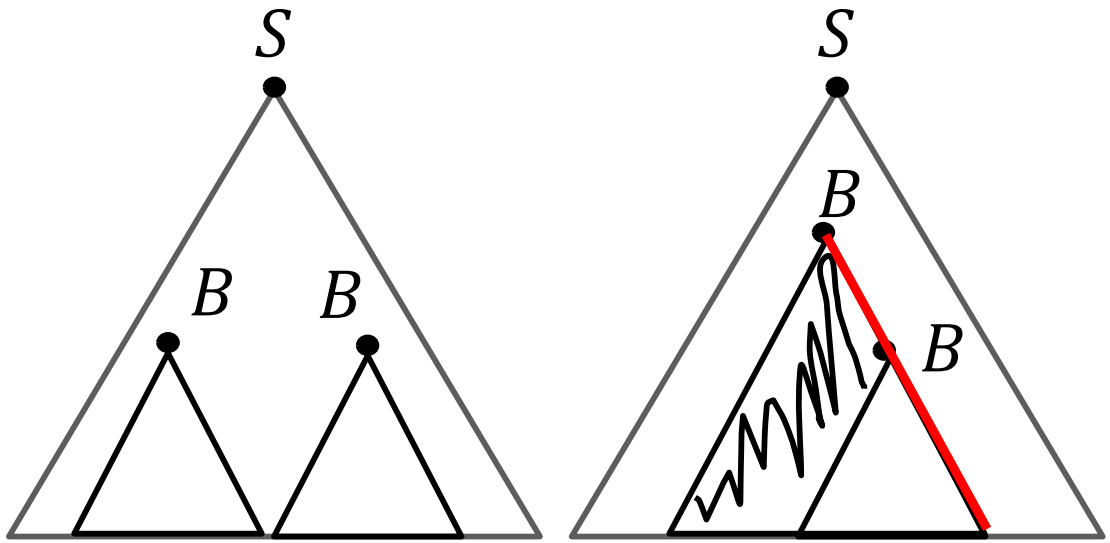
- For CFLs, we can have a parse tree for their strings, following derivation
- Internal nodes are variables, and leave nodes are terminals.



- For long enough strings, there must be **variables that are used twice**!
- In pumping lemma for regular languages, this already indicates **repetition**.
- Does it do the same for CFLs?

Pumping lemma for CFLs

- For long enough strings, there must be **variables that are used twice**, **but not necessarily there is repetition**.



No repetition in this case

Starting from B, we go through some variables and derivation, and we **still** arrive at B **=> this is what we can repeat!**

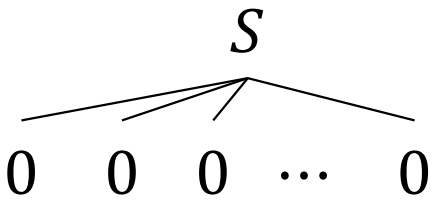
- The variables that appear multiple times should be on the **same path** from root to leaves.

Pumping lemma for CFLs

- How to guarantee that the repeated variables are on the same path and there is something we can repeat?
- Look back into the CFG and the parse tree

① there could be a long chain of rules which lead to empty string at the end, e.g., $A \to B, B \to C, C \to D, \dots, F \to \varepsilon$

② For a long string, we want a long path in the parsing tree, where we can guarantee there will be something we can repeat. => **we don't want a wide but short tree.**



Pumping lemma for CFLs: Chomsky Normal Form

- How to guarantee that the repeated variables are on the same path and there is something we can repeat?
- The **Chomsky Normal Form (CNF)**
- $S \to \varepsilon$, only the start variable can generate an empty string
- $A \to a$, a variable generates a terminal **or**
- $A \to BC$, a variable links to two variables
- In this way, we avoid multiple useless rules leading to empty string, and we limit that the children of variables are less than or equal to 2, **forcing a long path in the parse tree**.



Pumping lemma for CFLs: CNF

- As we want the pumping lemma to work for CFLs, so we need to first prove that: for each CFG, we can convert it to have a CNF
- **Step 1**: create a new start variable, pointing to the original one
- $S_0 \rightarrow S$
- **Step 2**: eliminate the ε-rules
- Only make ε appear at the RHS of S_0
- find all variables that originally lead to ε (i.e., nullable vars), remove these rules, and make changes to the remaining rules
- e.g., $S \rightarrow S_1 S \mid S_1 \mid \varepsilon$ can be changed to $S \rightarrow S_1 S \mid S_1$
- if a ε-rule makes no change to the generated string, delete it



Pumping lemma for CFLs: CNF

- As we want the pumping lemma to work for CFLs, so we need to first prove that: for each CFG, we can convert it to have a CNF
- **Step 3**: remove unit rules
- for all unit rules like $A \to B$, copy what B can generate and let A generate the same, i.e., $B \to ab \mid bc$ and $A \to B \mid a$, then we have $A \to ab \mid bc \mid cd \mid a$
- if $A \Rightarrow^* B$ and $B \to a$, a is a terminal, then add $A \to a$ and delete all unit rules related to $A \Rightarrow^* B$



Pumping lemma for CFLs: CNF

- As we want the pumping lemma to work for CFLs, so we need to first prove that: for each CFG, we can convert it to have a CNF
- **Step 4**: separate terminals
- if a rule has length >= 2 and contains a terminal a, like $A \to BCa$ create a new variable like $U_a \to a$, and the original rule becomes $A \to BCU_a$



Pumping lemma for CFLs: CNF

- As we want the pumping lemma to work for CFLs, so we need to first prove that: for each CFG, we can convert it to have a CNF
- **Step 5**: break long RHS of the rules
- For all rules with length >= 2, keep one original variable, and make new variables to store the rest, so that we create multiple new rules, but each rule **only contains two RHS variables**
- $A \rightarrow BCD$
- $A \rightarrow BC_1, C_1 \rightarrow CD$



Pumping lemma for CFLs: CNF

- $S \to aSb \mid A$
- $A \to a \mid \varepsilon$
- **Step 1**: create a new start variable, pointing to the original one
- $S_0 \to S$
- **Step 2**: eliminate the ε-rules
- Considering the original rules and the newly added rules, we now have three variables that can generate ε, i.e., $S_0 \Rightarrow S \Rightarrow A \Rightarrow \varepsilon$, $S \Rightarrow A \Rightarrow \varepsilon$, and $A \Rightarrow \varepsilon$
- We want to only keep $S_0 \to \varepsilon$, so remove the others.
- For $S \to aSb$, if $S \Rightarrow A \Rightarrow \varepsilon$, then we could directly have $S \to ab$



Pumping lemma for CFLs: CNF

- $S_0 \rightarrow S \mid \varepsilon$
- $S \rightarrow aSb \mid ab \mid A$
- $A \rightarrow a$
- **Step 3**: remove unit rules
- The unit rules are $S_0 \rightarrow S$ and $S \rightarrow \mathrm{A}$
- $S \rightarrow A$ only let S derive a, so $S \rightarrow aSb \mid ab \mid a$
- And hence we further have $S_0 \rightarrow aSb \mid ab \mid a \mid \varepsilon$



Pumping lemma for CFLs: CNF

- $S_0 \rightarrow aSb \mid ab \mid a \mid \varepsilon$
- $S \rightarrow aSb \mid ab \mid a$
- **Step 4**: Separate terminals
- Let $U_a \rightarrow a, and\ U_b \rightarrow b$
- $S_0 \rightarrow U_aSU_b \mid U_aU_b \mid a \mid \varepsilon$
- $S \rightarrow U_aSU_b \mid U_aU_b \mid a$
- **Step 5**: break long RHS of the rules
- Let $T \rightarrow SU_b$
- $S_0 \rightarrow U_aT \mid U_aU_b \mid a \mid \varepsilon$
- $S \rightarrow U_aT \mid U_aU_b \mid a$



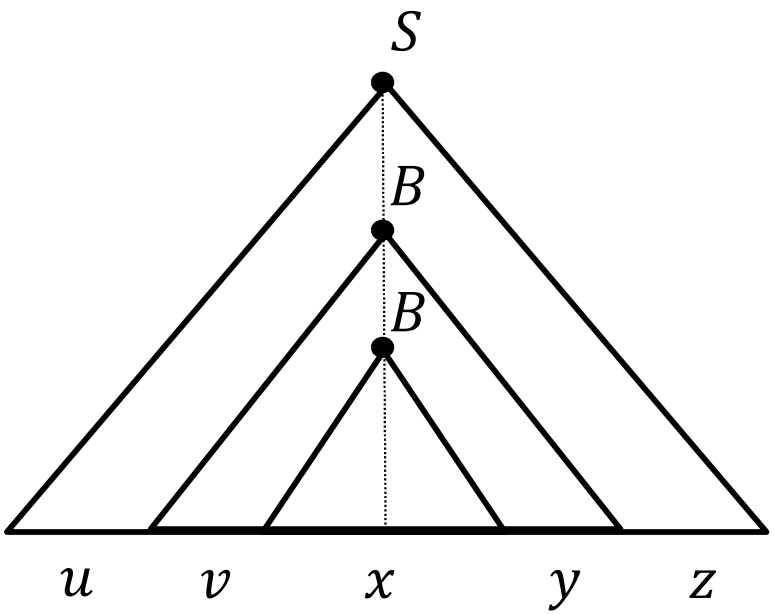
Pumping lemma for CFLs: CNF

- Then, for each CFL, we can convert its CFG to an equivalent CNF, and hence we could **guarantee** that a long enough string will have repeated variables in the middle
- Next, we define the **pumping lemma for CFLs**



Pumping lemma for CFLs

- Like the pumping lemma for regular languages, we can define that for a CFL L, there exists an integer $p \geq 1$, called the pumping length, such that for a string $w \in L$, with $|w| \geq p$, fulfills:



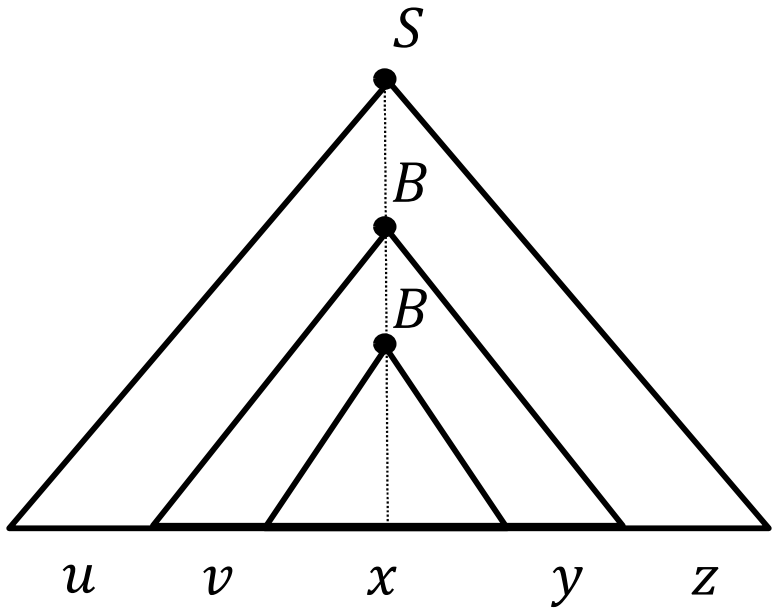
- $w = uvxyz$
- $uv^i xy^i z \in L, for\ all\ i \geq 0$
- v and y are the repeated part, i.e., the string generated by the trip from the top B to the down B.

Pumping lemma for CFLs

- Like the pumping lemma for regular languages, we can define:

For every CFL L, there is an integer p such that if $w \in L$ and $|w| \geq p$ then $w = uvxyz$ where:

- $uv^i xy^i z \in L, for\ all\ i \geq 0$
- $vy \neq \varepsilon$
- $|vxy| \leq p$



since w should be long enough, the extreme case is that **u and z are all empty**, so we have $|vxy| \leq p$

Example

- Prove that $L = \{0^n 1^n 2^n : n \geq 0\}$ is not context-free language.
- Suppose L is a CFL, there exists a p for L,
- say we choose $w = 0^p 1^p 2^p$
- Consider an arbitrary decomposition of $w = uvxyz$ with $|vxy| \leq p$ and $|vy| > 0$
- Case 1: v and y are only in 0's or 1's or 2's
- take 0 for example, $u = 0^a, v = 0^b, x = 0^c, y = 0^d, z = 0^{p-a-b-c-d} 1^p 2^p$
- let $i = 2$, we have $uv^2xy^2z = 0^{a+2b+c+2d+p-a-b-c-d} 1^p 2^p = 0^{p+b+d} 1^p 2^p$, since $|vy| > 0, p + b + d \neq p$, contradicts.



Example

- Prove that $L = \{0^n 1^n 2^n : n \geq 0\}$ is not context-free language.
- Suppose L is a CFL, there exists a p for L,
- say we choose $w = 0^p 1^p 2^p$
- Consider an arbitrary decomposition of $w = uvxyz$ with $|vxy| \leq p$ and $|vy| > 0$
- Case 1: v and y are only in 0's or 1's or 2's
- take 0 for example,
- assuming the number of added 0's after pumping up is *k*,
- we have $0^{p+k} 1^p 2^p$, $p + k \neq p$



Example

- Prove that $L = \{0^n 1^n 2^n : n \geq 0\}$ is not context-free language.
- Case 2: v and y are only in 0's and 1's or in 1's and 2's
- take 0's and 1's for example and pump up, we then have more 0's and 1's, and hence the number of 2's is less than that of 0's and 1's, leading to a contradiction.
- So L is not a CFL



Closure Properties of CFLs (Continued)

- **Recall that CFLs are closed under regular operations**
- Are CFLs closed under intersection or complement?



Closure Properties of CFLs (Continued)

- Intersection of CFLs
- $L_1 = \{0^n 1^n 2^k : n, k \geq 0\}$ and $L_2 = \{0^k 1^n 2^n : n, k \geq 0\}$ are two context-free languages
- $G_1 : S_1 \to AC, A \to aAb \mid \varepsilon, C \to cC \mid \varepsilon$
- view it as generating two strings independently and then concatenate them together
- but $L_1 \cap L_2 = \{0^n 1^n 2^n : n \geq 0\}$ is not CFL.



Closure Properties of CFLs (Continued)

- Complement of CFLs
- for language operations, we have a De Morgan law, where:
- $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$
- Assume that L_1 and L_2 are context-free
- we just prove **LHS is not context-free**
- if CFLs are closed under complement, then $\overline{L_1}$ and $\overline{L_2}$ are context-free
- CFLs are closed under union, so $\overline{L_1} \cup \overline{L_2}$ is context-free, and the whole **RHS is context-free => conflict**



Review for Today

- Each CFG has a CNF.
- Pumping lemma for CFLs can be used to prove that a language is not CFL.
- Proving a language is CFL needs to construct its CFGs or PDAs.



Review

- Regular languages
- Finite Automata
- Regexes
- Context-Free Grammars
- Context-Free Languages
- Pushdown Automata
- Pumping Lemma for CFLs



Q & A

① Can a rule in CNF look like $A \rightarrow aB$?

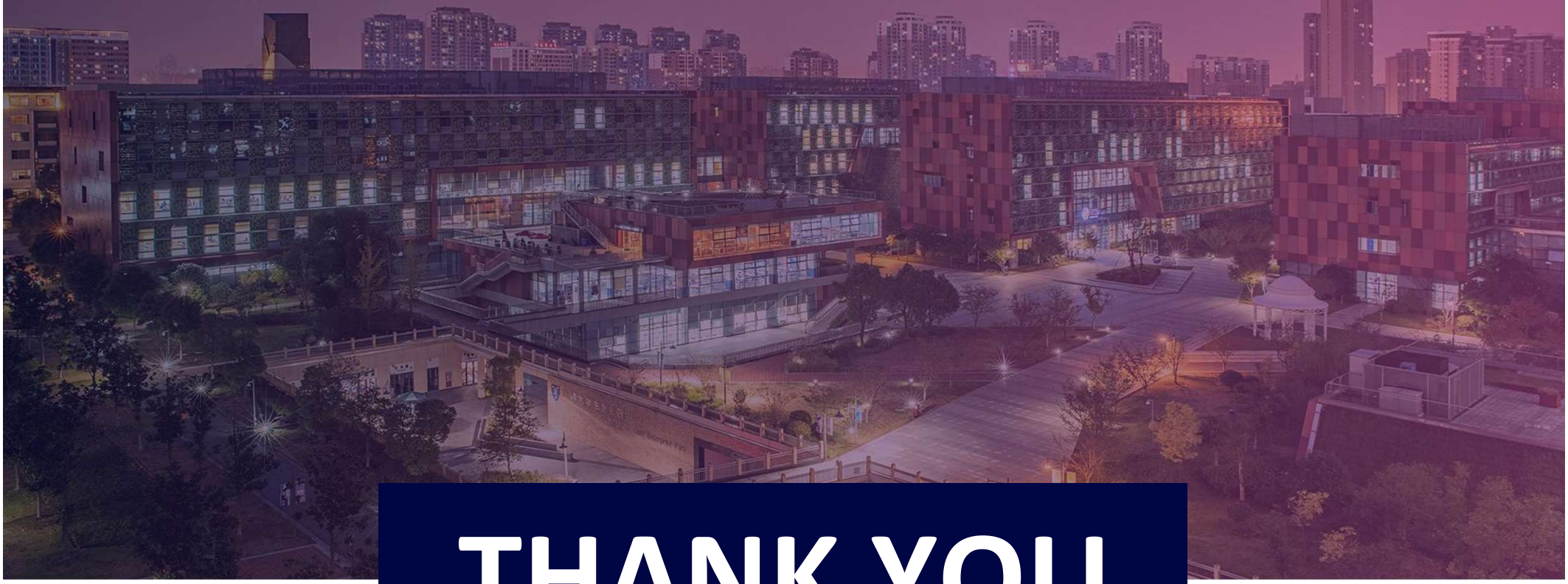
② Can we pump v without pumping y, or must they stay tied together?



Assistant Professor-Dr Xinhong Chen

- Graduated from City University of Hong Kong
  - Studied artificial intelligence and reinforcement learning
  - Research interests include *autonomous driving*, *causality analysis*, and *AI agents (agent self-evolution and multi-agent collaboration)*

- Contact: xinhong.chen@xjtlu.edu.cn
- WeChat: Mark_CXH
- Conduct interested research, SURF project, etc.



THANK YOU

Xi'an Jiaotong-Liverpool University
西交利物浦大学

XJTLU | SCHOOL OF FILM AND TV ARTS